

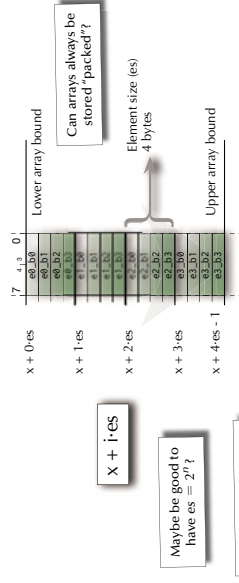


Data Structures

Uwe R. Zimmer - The Australian National University

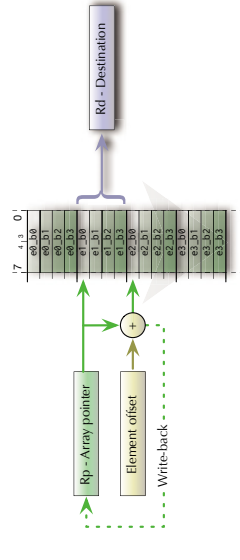
Data Structures

Array addressing



Data Structures

Array addressing via element pointer



`ld<C><q> <Rd>, [<Rp>], #+/-offset>`

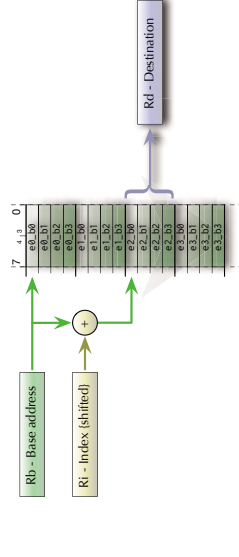
Data Structures

References for this chapter

[Patterson17]
David A. Patterson & John L. Hennessy
Computer Organization and Design – The Hardware/Software Interface
Chapter 2 "Instructions: Language of the Computer",
Chapter 5 "Large and Fast: Exploiting Memory Hierarchy"
AKM edition, Morgan Kaufmann 2017

Data Structures

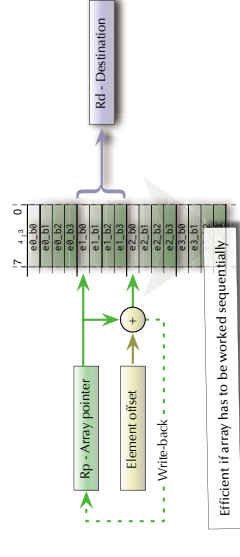
Array addressing via index register



`ld<C><q> <Rd>, [<Rb>, <Ri> {, 1s1 #shift}>]`

Data Structures

Array addressing via element pointer

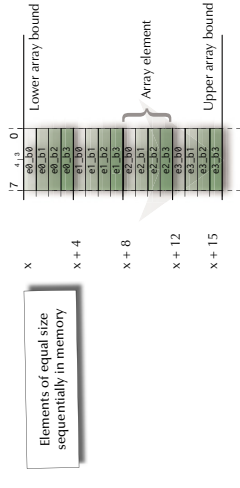


Efficient if array has to be worked sequentially

`ld<C><q> <Rd>, [<Rp>], #+/-offset>`

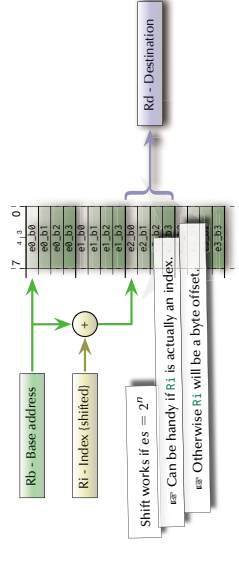
Data Structures

Array layout



Data Structures

Array addressing via index register



Shift works if $es = 2^n$

$\#R$ Can be handy if $R1$ is actually an index.

$\#R$ Otherwise $R1$ will be a byte offset.

`ld<C><q> <Rd>, [<Rb>, <Ri> {, 1s1 #shift}>]`

Data Structures

Calculate $\sum_i x_i$

```
int sum (unsigned int uints [], unsigned int from, unsigned int to) {
    int i;
    int acc = 0;
    for (i = from; i <= to; i++) {
        acc += uints [i];
    }
    return acc;
}
```

type `Naturals` is array (Integer range $\leftrightarrow$) of `Natural`;
function `Sum (Naturals : Naturals)` return `Natural` is

Acc : `Natural` := 0;

begin

for n of `Naturals` loop

Acc := Acc + n;

end loop;

return Acc;

end Sum;

`Naturals` := constant `Naturals` (1 .. 100) :=
(others => Random (Numbers_Generator));
`Sum_of_Naturals` := Sum (`Naturals`);

Data Structures

Calculate $\sum x_i$

```
int sum (unsigned int uints [], unsigned int from, unsigned int to) {
    int i;
    int acc = 0;
    for (i = from; i <= to; i++) {
        unsigned int s;
        acc += uints [i];
    }
    return acc;
}
```

Best of luck with the array bounds!

```
type Natural is array (Integer range <>) of Natural;
function Sum (Numbers : Natural) return Natural is
```

```
Acc : Natural := 0;
```

```
begin
    for n of Numbers loop
```

```
        Acc := Acc + n;
```

```
    end loop;
```

```
    return Acc;
```

```
end Sum;
```

Numbers : constant Natural (1 .. 100) :=

Sum_of_Natural (others => Random (Numbers.Generator));

Sum_of_Natural : constant Natural := Sum (Numbers);

Data Structures

Replacing multiplication with shifted index register

```
; r0 base address for array a
; r1 from array index
; r2 to array index
```

```
mov r3, #0 ; sum := 0
for_sum:
    cmp r1, r2 ; i > to
    bgt end_for_sum
    ldr r4, [r0, r1, #2] ; a [i] := [base + element_offset]
    add r3, r4 ; sum := sum + a [i]
    add r1, #1 ; i := i + 1
    b for_sum
end_for_sum:
    mov r0, r3 ; r0 sum over all a [from .. to]
```

Data Structures

Replacing offsets with addresses

```
; r0 base address for array a
; r1 from array index
; r2 to array index
```

```
lsl r1, r1, #2 ; translate from index to offset
lsl r2, r2, #2 ; translate from index to offset
add r1, r0 ; translate from index to address -> i_addr
add r2, r0 ; translate from index to address -> to_addr
mov r0, #0 ; sum := 0
for_sum:
    ldr r3, [r1, #4] ; a [i] := [i_addr]; i_addr += 4
    add r0, r3 ; sum := sum + a [i]
    cmp r1, r2 ; i_addr <= to_addr
    ble for_sum
end_for_sum:
    ; r0 sum over all a [from .. to]
```

Data Structures

Arbitrary array indexing

```
; r0 base address for array a
; r1 from array index
; r2 to array index
```

```
mov r3, #0 ; sum := 0
mov r4, #4 ; element size is 4 bytes
mov r5, #1 ; firstElement_offset
for_sum:
    cmp r1, r2 ; i > to
    bgt end_for_sum
    mla r0, r1, r4, r5 ; element_offset := (i * 4) - firstElement_offset
    ldr r7, [r0, r6] ; a [i] := [base + element_offset]
    add r3, r7 ; sum := sum + a [i]
    add r1, #1 ; i := i + 1
    b for_sum
end_for_sum:
    mov r0, r3 ; r0 sum over all a [from .. to]
```

Data Structures

Replacing indices with offsets

```
; r0 base address for array a
; r1 from array index
; r2 to array index
```

```
lsl r1, r1, #2 ; translate from index to offset
lsl r2, r2, #2 ; translate from index to offset
mov r3, #0 ; sum := 0
for_sum:
    cmp r1, r2 ; i > to
    bgt end_for_sum
    ldr r4, [r0, r1] ; a [i] := [base + offset]
    add r3, r4 ; sum := sum + a [i]
    add r1, #4 ; offset := offset + 4
    b for_sum
end_for_sum:
    mov r0, r3 ; r0 sum over all a [from .. to]
```

Data Structures

Array Slices

```
numbers
    = [0, 1, 2, 3, 4, 5]
numbersSlice = numbers [1:3]
# numbersSlice equals [1, 2, 3]
```

```
numbers := [int (0, 1, 2, 3, 4, 5)]
numbersSlice := numbers [1:3]
type Natural is array (Integer range <>) of Natural;
Numbers : constant Natural (-50 .. 50) := (others => Random (Generator));
Numbers.Slice_1 : constant Natural
Numbers.Slice_2 : constant Natural ( 1 .. 10) := Numbers (-10 .. 10);
Numbers.Slice_3 : constant Natural
                := Numbers (-20 .. 50);
begin
    for n of Numbers.Slice_3 loop
        n := n + 1;
    end loop;
end;
```

Data Structures

Zero-based array indexing

```
; r0 base address for array a
; r1 from array index
; r2 to array index
```

```
mov r3, #0 ; sum := 0
mov r4, #4 ; element size is 4 bytes
for_sum:
    cmp r1, r2 ; i > to
    bgt end_for_sum
    mul r5, r1, r4 ; element_offset := (i * 4)
    ldr r6, [r0, r5] ; a [i] := [base + element_offset]
    add r3, r6 ; sum := sum + a [i]
    add r1, #1 ; i := i + 1
    b for_sum
end_for_sum:
    mov r0, r3 ; r0 sum over all a [from .. to]
```

Data Structures

Assuming non-empty arrays

```
; r0 base address for array a
; r1 from array index
; r2 to array index
```

```
lsl r1, r1, #2 ; translate from index to offset
lsl r2, r2, #2 ; translate from index to offset
mov r3, #0 ; sum := 0
for_sum:
    ldr r4, [r0, r1] ; a [i] := [base + offset]
    add r3, r4 ; sum := sum + a [i]
    cmp r1, #4 ; offset := offset + 4
    cmp r1, r2 ; i <= to
    ble for_sum
end_for_sum:
    mov r0, r3 ; r0 sum over all a [from .. to]
```

Data Structures

Array Slices

```
numbers
    = [0, 1, 2, 3, 4, 5]
numbersSlice = numbers [1:3]
# numbersSlice equals [1, 2, 3]
```

```
numbers := [int (0, 1, 2, 3, 4, 5)]
numbersSlice := numbers [1:3]
type Natural is array (Integer range <>) of Natural;
Numbers : constant Natural (-50 .. 50) := (others => Random (Generator));
Numbers.Slice_1 : constant Natural
Numbers.Slice_2 : constant Natural ( 1 .. 10) := Numbers (-10 .. 10);
Numbers.Slice_3 : constant Natural
                := Numbers (-20 .. 50);
begin
    for n of Numbers.Slice_3 loop
        n := n + 1;
    end loop;
end;
```



Data Structures

Copy array slice

```
; r0 base address for array a
; r1 from array index
; r2 to array index >= from index
; r3 base address for array b

lsl r1, r1, #2
lsl r2, r2, #2
add r1, r0
add r2, r0

for_copy:
ldr r4, [r1], #4
str r4, [r3], #4
cmp r1, r2
ble for_copy

end_for_copy:
; a [i] := [l_addr]; l_addr += 4
; l_addr := a [i] ; j_addr += 4
; l_addr <= to_addr
; b [j] := a [from .. to]
```

4

6

© 2019 Awa K. Zimmer, The Australian National University

page 245 of 481 Chapter 4 "Data Structures" 40 to page 247



Data Structures

Copy array slice

```
; r0 base address for array a
; r1 from array index
; r2 to array index >= from index
; r3 base address for array b

lsl r1, r1, #2
lsl r2, r2, #2
add r1, r0
add r2, r0

for_copy:
ldr r4, [r1], #4
str r4, [r3], #4
cmp r1, r2
ble for_copy

end_for_copy:
; a [i] := [l_addr]; l_addr += 4
; l_addr := a [i] ; j_addr += 4
; l_addr <= to_addr
; b [j] := a [from .. to]
```

4

6

© 2019 Awa K. Zimmer, The Australian National University

page 246 of 481 Chapter 4 "Data Structures" 40 to page 247

Moving blocks of memory can be done even/much faster with special hardware.

eg DMA controllers



Data Structures

Summary

Data Structures

- Arrays
 - Structure
 - Alignment
 - Addressing
 - Iterators
 - Copy procedures

© 2019 Awa K. Zimmer, The Australian National University

page 247 of 481 Chapter 4 "Data Structures" 40 to page 247